

第九章 单元测试（面试突击★★★）

单元测试是软件质量的基石。本章覆盖 Google Test 核心用法、Mock、参数化测试、覆盖率、CI 集成和最佳实践。

目录

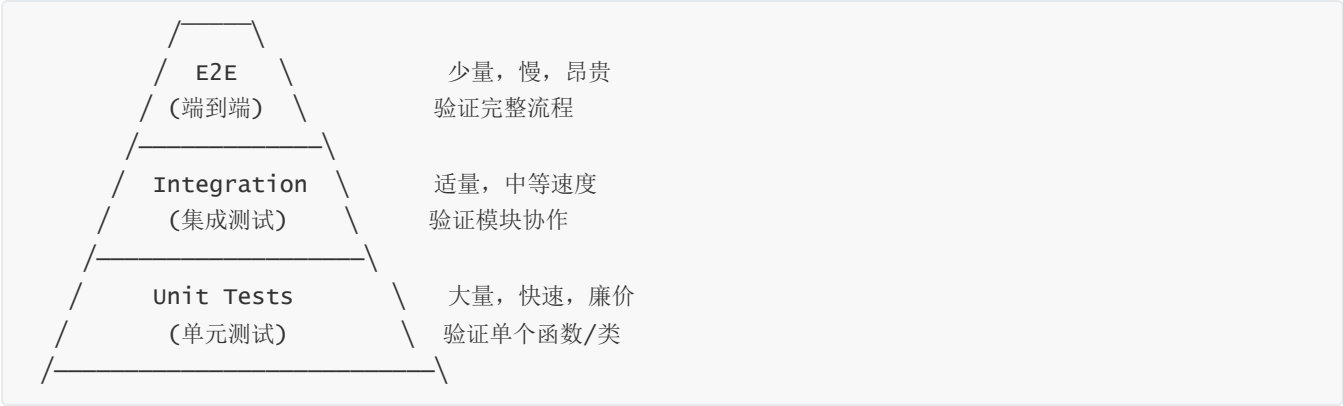
- 1. [测试基础](#)
- 2. [Google Test 框架](#)
- 3. [断言 \(Assertions\)](#)
- 4. [Test Fixture \(测试夹具\)](#)
- 5. [Google Mock](#)
- 6. [参数化测试](#)
- 7. [代码覆盖率](#)
- 8. [CI 集成](#)
- 9. [测试最佳实践](#)
- 10. [面试核心](#)

1. 测试基础

1.1 为什么要写单元测试

价值	说明
防止回归	修改代码后快速发现是否破坏了已有功能
文档行为	测试 = 可执行的需求文档，描述代码应该做什么
设计反馈	难测试 = 代码耦合度高，测试驱动更好的设计
信心保障	重构时有安全网，放心改
加速开发	减少手动验证时间，CI 自动跑
降低 bug 修复成本	Bug 发现越早，修复成本越低

1.2 测试金字塔



面试金句: "测试金字塔的核心思想是：底层（单元测试）应该最厚最多，越上层越少。单元测试快且稳定，E2E 测试慢且脆弱，所以大多数测试应该在单元层。"

1.3 好的测试应该具备 F.I.R.S.T

特性	说明
Fast (快速)	一个测试在毫秒级完成，整个 suite 在秒级完成
Independent (独立)	测试之间不依赖，可以任意顺序运行
Repeatable (可重复)	每次运行结果一致，不依赖外部环境
Self-validating (自验证)	自动判断通过/失败，不需要人工检查
Timely (及时)	在代码之前（TDD）或与代码同时编写

2. Google Test 框架

2.1 项目配置 (CMakeLists.txt)

```
cmake_minimum_required(VERSION 3.14)
project(MyProject)

# 方案 A: 使用 FetchContent (推荐)
include(FetchContent)

FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG        v1.15.2
)
FetchContent_MakeAvailable(googletest)

# 方案 B: 系统安装后使用 find_package
# find_package(GTest REQUIRED)
```

```

# 启用测试
enable_testing()

# 添加测试可执行文件
add_executable(
    my_tests
    tests/test_main.cpp
    tests/test_calculator.cpp
    tests/test_user_service.cpp
)

target_link_libraries(my_tests PRIVATE
    GTest::gtest_main      # 自动提供 main()
    GTest::gmock_main      # 如果需要 Mock
    my_library             # 被测试的库
)

# 注册到 CTest
include(GoogleTest)
gtest_discover_tests(my_tests)

```

2.2 基本测试结构

```

#include <gtest/gtest.h>

// TEST(TestSuiteName, TestName)
// ---> 命名建议: Should_ExpectedBehavior_When_Condition

// 最简单的测试
TEST(BasicTest, AdditionWorks) {
    EXPECT_EQ(1 + 1, 2);
}

// 测试一个函数
int Add(int a, int b) { return a + b; }

TEST(AddTest, ShouldReturnSum_WhenTwoPositiveNumbers) {
    EXPECT_EQ(Add(2, 3), 5);
    EXPECT_EQ(Add(10, 20), 30);
}

TEST(AddTest, ShouldHandleNegativeNumbers) {
    EXPECT_EQ(Add(-2, 3), 1);
    EXPECT_EQ(Add(-5, -5), -10);
}

TEST(AddTest, ShouldHandleZero) {
    EXPECT_EQ(Add(0, 0), 0);
    EXPECT_EQ(Add(5, 0), 5);
    EXPECT_EQ(Add(0, 5), 5);
}

```

3. 断言 (Assertions)

3.1 ASSERT vs EXPECT 区别

宏	失败时行为	使用场景
EXPECT_*	记录失败，继续执行	大部分场景（获取更多失败信息）
ASSERT_*	记录失败， 立即返回	后续代码依赖此断言成功（如指针非空才能解引用）

3.2 显式布尔断言

```
TEST(AssertionTest, ExplicitBoolean) {
    bool condition = true;

    ASSERT_TRUE(condition);    // 断言为真
    EXPECT_TRUE(condition);

    ASSERT_FALSE(false);      // 断言为假
    EXPECT_FALSE(false);
}
```

3.3 比较断言

```
TEST(AssertionTest, ComparisonAssertions) {
    // EXPECT_EQ(expected, actual) - 相等
    EXPECT_EQ(4, 2 + 2);
    EXPECT_EQ(std::string("hello"), std::string("hello"));

    // EXPECT_NE - 不等
    EXPECT_NE(5, 2 + 2);

    // EXPECT_LT - 小于
    EXPECT_LT(3, 5);

    // EXPECT_LE - 小于等于
    EXPECT_LE(5, 5);
    EXPECT_LE(4, 5);

    // EXPECT_GT - 大于
    EXPECT_GT(5, 3);

    // EXPECT_GE - 大于等于
    EXPECT_GE(5, 5);
    EXPECT_GE(6, 5);
}
```

3.4 字符串断言

```
#include <gtest/gtest.h>
#include <string>

TEST(AssertionTest, StringAssertions) {
    std::string str1 = "hello";
    std::string str2 = "hello";

    // C++ string 用 EXPECT_EQ
    EXPECT_EQ(str1, str2);

    // C 风格字符串用 EXPECT_STREQ (比较内容, 非指针)
    const char* cstr1 = "world";
    const char* cstr2 = "world";
    EXPECT_STREQ(cstr1, cstr2);    // 比较内容
    EXPECT_STRNE(cstr1, "hello"); // 内容不等

    // EXPECT_STRCASEEQ - 忽略大小写比较
    EXPECT_STRCASEEQ("HELLO", "hello");

    // 字符串包含检查
    EXPECT_STRNE(str1.c_str(), nullptr); // 不是 nullptr
}
```

3.5 浮点数断言

```
#include <cmath>

TEST(AssertionTest, FloatingPoint) {
    double a = 0.1 + 0.2;
    double b = 0.3;

    // 不要用 EXPECT_EQ 比较浮点数!
    // EXPECT_EQ(a, b); // 可能失败: 0.1+0.2 != 0.3 in floating point

    // 使用 EXPECT_NEAR(expected, actual, tolerance)
    EXPECT_NEAR(a, b, 1e-9);

    // EXPECT_DOUBLE_EQ - 默认 4 ULPs 容差
    EXPECT_DOUBLE_EQ(0.3, 0.1 + 0.2);

    EXPECT_FLOAT_EQ(0.3f, 0.1f + 0.2f);
}
```

3.6 异常断言

```
#include <stdexcept>

int Divide(int a, int b) {
    if (b == 0) throw std::invalid_argument("division by zero");
    return a / b;
}

void MightThrow(bool shouldThrow) {
    if (shouldThrow) throw std::runtime_error("bad");
}

TEST(AssertionTest, ExceptionAssertions) {
    // EXPECT_THROW - 期望抛出指定类型异常
    EXPECT_THROW(Divide(10, 0), std::invalid_argument);

    // EXPECT_NO_THROW - 期望不抛出异常
    EXPECT_NO_THROW(Divide(10, 2));

    // EXPECT_ANY_THROW - 期望抛出任意异常
    EXPECT_ANY_THROW(MightThrow(true));
    EXPECT_NO_THROW(MightThrow(false));

    // 检查异常消息
    EXPECT_THROW({
        try {
            Divide(10, 0);
        } catch (const std::invalid_argument& e) {
            EXPECT_STREQ("division by zero", e.what());
            throw; // 重新抛出让 EXPECT_THROW 捕获
        }
    }, std::invalid_argument);
}
```

3.7 死亡测试 (Death Tests)

```
#include <cassert>

void DieIfNegative(int x) {
    assert(x >= 0); // 违反时 abort()
}

TEST(DeathTest, ShouldAbort_whenNegativeInput) {
    // ASSERT_DEATH - 期望以某种方式终止
    ASSERT_DEATH(DieIfNegative(-1), "");

    // ASSERT_DEATH_IF_SUPPORTED - 在不支持的平台不失败
    ASSERT_DEATH_IF_SUPPORTED(DieIfNegative(-1), "");

    // 检查退出码
```

```

    ASSERT_EXIT(exit(42), ::testing::ExitedWithCode(42), "");
}

```

3.8 断言完整速查表

// 断言宏	用途	
// ASSERT_TRUE(c)	c 为 true	
// ASSERT_FALSE(c)	c 为 false	
// ASSERT_EQ(a,b)	a == b (支持 << 自定义输出)	
// ASSERT_NE(a,b)	a != b	
// ASSERT_LT(a,b)	a < b	
// ASSERT_LE(a,b)	a <= b	
// ASSERT_GT(a,b)	a > b	
// ASSERT_GE(a,b)	a >= b	
// ASSERT_STREQ(a,b)	C 字符串内容相等	
// ASSERT_STRNE(a,b)	C 字符串内容不等	
// ASSERT_STRCASEEQ	C 字符串忽略大小写相等	
// ASSERT_FLOAT_EQ	float 相等 (4 ULPs)	
// ASSERT_DOUBLE_EQ	double 相等 (4 ULPs)	
// ASSERT_NEAR(a,b,t)	a-b <= t	
// ASSERT_THROW(s,e)	s 抛出 e 类型异常	
// ASSERT_NO_THROW(s)	s 不抛出异常	
// ASSERT_ANY_THROW(s)	s 抛出任意异常	
// ASSERT_DEATH(s,r)	s 以匹配 r 的方式终止	
// EXPECT_*	同上, 失败后继续执行	

3.9 自定义失败消息

```

TEST(AssertionTest, CustomMessage) {
    int actual = 42;
    int expected = 100;

    EXPECT_EQ(actual, expected) << "value mismatch at index "
                                << 5 << ", expected " << expected
                                << " but got " << actual;
    // 输出: value mismatch at index 5, expected 100 but got 42
}

```

4. Test Fixture (测试夹具)

4.1 基本 Fixture: SetUp / TearDown

```
#include <gtest/gtest.h>
#include <vector>
#include <memory>

// 被测类
class Stack {
public:
    void Push(int value) { data_.push_back(value); }
    int Pop() {
        if (data_.empty()) throw std::out_of_range("empty");
        int val = data_.back();
        data_.pop_back();
        return val;
    }
    int Top() const {
        if (data_.empty()) throw std::out_of_range("empty");
        return data_.back();
    }
    size_t Size() const { return data_.size(); }
    bool IsEmpty() const { return data_.empty(); }
private:
    std::vector<int> data_;
};

// Fixture: 每个 TEST_F 前调用 SetUp, 后调用 TearDown
class StackTest : public ::testing::Test {
protected:
    // SetUp - 每个测试前执行
    void SetUp() override {
        stack_ = std::make_unique<Stack>();
        // 预填充一些数据
        stack_>Push(1);
        stack_>Push(2);
        stack_>Push(3);
    }

    // TearDown - 每个测试后执行
    void TearDown() override {
        // 清理工作 (unique_ptr 自动释放, 这里作为示范)
        stack_.reset();
    }

    // 辅助方法 - 供测试用例使用
    void PushMany(size_t count) {
        for (size_t i = 0; i < count; ++i) {
            stack_>Push(static_cast<int>(i + 10));
        }
    }
};
```



```

    std::unique_ptr<Stack> stack_;
};

// 使用 TEST_F 而非 TEST
TEST_F(StackTest, ShouldHaveThreeElements_AfterSetUp) {
    EXPECT_EQ(stack_>Size(), 3);
    EXPECT_EQ(stack_>Top(), 3);
}

TEST_F(StackTest, ShouldReturnTop_whenPop) {
    int val = stack_>Pop();
    EXPECT_EQ(val, 3);
    EXPECT_EQ(stack_>Size(), 2);
    EXPECT_EQ(stack_>Top(), 2);
}

TEST_F(StackTest, ShouldThrow_whenPopEmpty) {
    stack_>Pop(); // 3
    stack_>Pop(); // 2
    stack_>Pop(); // 1
    EXPECT_TRUE(stack_>IsEmpty());
    EXPECT_THROW(stack_>Pop(), std::out_of_range);
}

TEST_F(StackTest, ShouldIncreaseSize_whenPush) {
    size_t oldSize = stack_>Size();
    stack_>Push(42);
    EXPECT_EQ(stack_>Size(), oldSize + 1);
    EXPECT_EQ(stack_>Top(), 42);
}

TEST_F(StackTest, ShouldBeEmpty_whenNewStack) {
    Stack emptyStack;
    EXPECT_TRUE(emptyStack.IsEmpty());
    EXPECT_EQ(emptyStack.Size(), 0);
}

```

4.2 静态 Fixture: SetUpTestCase / TearDownTestCase

```

// 静态 Fixture - 整个 test suite 共享的昂贵资源
class DatabaseTest : public ::testing::Test {
protected:
    // 所有测试开始前执行一次 (注意是 static)
    static void SetUpTestSuite() {
        // 启动内存数据库 (耗时操作, 只做一次)
        db_ = new MockDatabase();
        db_>Connect();
        db_>LoadSchema();

        std::cout << "[SUITE] Database initialized once" << std::endl;
    }
}

```

```

// 所有测试结束后执行一次（注意是 static）
static void TearDownTestSuite() {
    db_>Disconnect();
    delete db_;
    db_ = nullptr;

    std::cout << "[SUITE] Database cleaned up" << std::endl;
}

// 每个测试前后仍可用 SetUp / TearDown
void SetUp() override {
    // 每个测试前开启事务
    db_>BeginTransaction();
}

void TearDown() override {
    // 每个测试后回滚，保证测试隔离
    db_>RollbackTransaction();
}

static MockDatabase* db_;
};

MockDatabase* DatabaseTest::db_ = nullptr;

TEST_F(DatabaseTest, ShouldInsertRecord_whenValidData) {
    bool ok = db_>Insert("users", "name=Alice");
    EXPECT_TRUE(ok);
}

TEST_F(DatabaseTest, ShouldReturnError_whenDuplicateKey) {
    db_>Insert("users", "id=1,name=Alice");
    bool ok = db_>Insert("users", "id=1,name=Bob");
    EXPECT_FALSE(ok);
}

```

5. Google Mock

5.1 Mock 基础概念

Mock 用于模拟外部依赖，使单元测试真正“单元”化。

```

#include <gmock/gmock.h>

// 被测代码依赖的接口
class IFileSystem {
public:
    virtual ~IFileSystem() = default;
    virtual bool FileExists(const std::string& path) const = 0;
    virtual std::string ReadFile(const std::string& path) = 0;

```

```

    virtual void writeFile(const std::string& path,
                           const std::string& content) = 0;
    virtual std::vector<std::string> ListFiles(
        const std::string& directory) = 0;
};

// 实际的实现（生产环境）
class RealFileSystem : public IFileSystem {
public:
    bool FileExists(const std::string& path) const override {
        return std::filesystem::exists(path);
    }
    std::string ReadFile(const std::string& path) override {
        std::ifstream in(path);
        return std::string(std::istreambuf_iterator<char>(in),
                           std::istreambuf_iterator<char>());
    }
    void writeFile(const std::string& path,
                   const std::string& content) override {
        std::ofstream out(path);
        out << content;
    }
    std::vector<std::string> ListFiles(
        const std::string& directory) override {
        std::vector<std::string> result;
        for (const auto& entry :
             std::filesystem::directory_iterator(directory)) {
            result.push_back(entry.path().filename().string());
        }
        return result;
    }
};

// Mock 实现（测试环境）
class MockFileSystem : public IFileSystem {
public:
    // MOCK_METHOD(返回类型, 函数名, (参数列表), (修饰符))
    MOCK_METHOD(bool, FileExists, (const std::string& path),
                 (const, override));
    MOCK_METHOD(std::string, ReadFile, (const std::string& path),
                 (override));
    MOCK_METHOD(void, writeFile,
                 (const std::string& path, const std::string& content),
                 (override));
    MOCK_METHOD(std::vector<std::string>, ListFiles,
                 (const std::string& directory), (override));
};

```

5.2 EXPECT_CALL 详解

```
using ::testing::Return;
using ::testing::_;
using ::testing::AtLeast;
using ::testing::Exactly;
using ::testing::AtMost;
using ::testing::Times;
using ::testing::WillOnce;
using ::testing::WillRepeatedly;

TEST(MockTest, BasicExpectCall) {
    MockFileSystem mock;

    // 期望 FileExists 被调用至少 1 次, 参数任意, 返回 true
    EXPECT_CALL(mock, FileExists(_))
        .Times(AtLeast(1))
        .WillRepeatedly(Return(true));

    // 期望 ReadFile("config.json") 被精确调用 1 次, 返回特定内容
    EXPECT_CALL(mock, ReadFile("config.json"))
        .Times(Exactly(1))
        .WillOnce(Return("{\"port\": 8080}"));

    // 期望 writeFile 被调用, 用 willOnce 返回不同值
    EXPECT_CALL(mock, ReadFile("data.txt"))
        .WillOnce(Return("data1"))
        .WillOnce(Return("data2"));

    // 使用 mock
    EXPECT_TRUE(mock.FileExists("anything.txt"));
    EXPECT_EQ(mock.ReadFile("config.json"), "{\"port\": 8080}");
    EXPECT_EQ(mock.ReadFile("data.txt"), "data1");
    EXPECT_EQ(mock.ReadFile("data.txt"), "data2");
}
```

5.3 ON_CALL vs EXPECT_CALL

```
// ON_CALL: 定义默认行为 (不关心调用次数)
// EXPECT_CALL: 定义期望行为 (关心调用情况)

TEST(MockTest, OnCallVsExpectCall) {
    MockFileSystem mock;

    // ON_CALL - 设置默认行为 (stub), 不强制必须调用
    ON_CALL(mock, FileExists(_)).WillByDefault(Return(true));

    // EXPECT_CALL - 期望被调用, 如果没调用测试失败
    EXPECT_CALL(mock, ReadFile("test.txt"))
        .Times(1)
        .WillOnce(Return("content"));
}
```

```

// 不调用也 OK (没有 EXPECT_CALL 对 FileExists 的约束)
mock.ReadFile("test.txt"); // 必须调用, 否则失败
}

```

5.4 完整的 Mock 示例：数据库接口

```

#include <gmock/gmock.h>

// ----- 接口 -----
struct User {
    int id;
    std::string name;
    std::string email;
};

class IUserRepository {
public:
    virtual ~IUserRepository() = default;
    virtual std::optional<User> FindById(int id) = 0;
    virtual std::vector<User> FindAll() = 0;
    virtual bool Save(const User& user) = 0;
    virtual bool Delete(int id) = 0;
};

// ----- Mock -----
class MockUserRepository : public IUserRepository {
public:
    MOCK_METHOD(std::optional<User>, FindById, (int id), (override));
    MOCK_METHOD(std::vector<User>, FindAll, (), (override));
    MOCK_METHOD(bool, Save, (const User& user), (override));
    MOCK_METHOD(bool, Delete, (int id), (override));
};

// ----- 被测类 -----
class UserService {
public:
    explicit UserService(std::shared_ptr<IUserRepository> repo)
        : repo_(std::move(repo)) {}

    std::string GetUserDisplayName(int id) {
        auto user = repo_>FindById(id);
        if (!user.has_value()) return "Unknown User";
        return user->name + " <" + user->email + ">";
    }

    bool CreateUser(const std::string& name, const std::string& email) {
        if (name.empty() || email.empty()) return false;
        User user{0, name, email};
        return repo_>Save(user);
    }
}

```

```

    bool TransferUser(int fromId, int toId) {
        auto from = repo_>FindById(fromId);
        auto to = repo_>FindById(toId);
        if (!from.has_value() || !to.has_value()) return false;
        // 合并逻辑 ...
        return repo_>Save(from.value()) && repo_>Save(to.value());
    }

private:
    std::shared_ptr<IUserRepository> repo_;
};

// ----- 测试 -----
class UserServiceTest : public ::testing::Test {
protected:
    void SetUp() override {
        mockRepo_ = std::make_shared<MockUserRepository>();
        service_ = std::make_unique<UserService>(mockRepo_);
    }

    std::shared_ptr<MockUserRepository> mockRepo_;
    std::unique_ptr<UserService> service_;
};

TEST_F(UserServiceTest, ShouldReturnFormattedName_whenUserExists) {
    User alice{1, "Alice", "alice@example.com"};

    EXPECT_CALL(*mockRepo_, FindById(1))
        .willOnce(Return(alice));

    auto result = service_>GetUserDisplayName(1);
    EXPECT_EQ(result, "Alice <alice@example.com>");
}

TEST_F(UserServiceTest, ShouldReturnUnknown_whenUserNotFound) {
    EXPECT_CALL(*mockRepo_, FindById(999))
        .willOnce(Return(std::nullopt));

    auto result = service_>GetUserDisplayName(999);
    EXPECT_EQ(result, "Unknown User");
}

TEST_F(UserServiceTest, ShouldReturnFalse_whenCreateWithEmptyName) {
    // Save 不应该被调用
    EXPECT_CALL(*mockRepo_, Save(::testing::_)).Times(0);

    bool result = service_>CreateUser("", "alice@example.com");
    EXPECT_FALSE(result);
}

TEST_F(UserServiceTest, ShouldSaveUser_whenCreateWithValidData) {
    EXPECT_CALL(*mockRepo_, Save(::testing::_))
        .willOnce(Return(true));
}

```

```

    bool result = service_>CreateUser("Alice", "alice@example.com");
    EXPECT_TRUE(result);
}

TEST_F(UserServiceTest, ShouldCallSaveTwice_whenTransferUser) {
    User from{1, "Alice", "a@e.com"};
    User to{2, "Bob", "b@e.com"};

    EXPECT_CALL(*mockRepo_, FindById(1)).WillOnce(Return(from));
    EXPECT_CALL(*mockRepo_, FindById(2)).WillOnce(Return(to));
    EXPECT_CALL(*mockRepo_, Save(::testing::_))
        .Times(2)
        .WillRepeatedly(Return(true));

    bool result = service_>TransferUser(1, 2);
    EXPECT_TRUE(result);
}

```

5.5 Mock 进阶：参数匹配器

```

using ::testing::_;
using ::testing::Ge;
using ::testing::Lt;
using ::testing::HasSubstr;
using ::testing::StartsWith;
using ::testing::EndsWith;
using ::testing::AllOf;
using ::testing::AnyOf;
using ::testing::Not;
using ::testing::Field;
using ::testing::Property;

TEST(MockTest, ArgumentMatchers) {
    MockUserRepository mock;

    // 通配符 - 匹配任意参数
    EXPECT_CALL(mock, Save(_)).WillOnce(Return(true));
    mock.Save(User{1, "A", "a@b.com"});

    // 范围匹配
    EXPECT_CALL(mock, FindById(Ge(5))) // id >= 5
        .WillRepeatedly(Return(std::nullopt));
    EXPECT_CALL(mock, FindById(Lt(5))) // id < 5
        .WillRepeatedly(Return(User{1, "A", ""}));

    // 字符串匹配
    EXPECT_CALL(mock, Save(Field(&User::name, HasSubstr("test"))))
        .WillOnce(Return(true));

    // 组合匹配
    EXPECT_CALL(mock, Save(AllOf(

```

```

        Field(&User::name, Startswith("VIP_")),
        Field(&User::id, Ge(1000))
    )))WillOnce(Return(true));
}

```

5.6 Invoke: 自定义 Mock 行为

```

#include <queue>

TEST(MockTest, InvokeCustomBehavior) {
    MockUserRepository mock;

    std::queue<std::optional<User>> results;
    results.push(User{1, "Alice", "alice@example.com"});
    results.push(std::nullopt);
    results.push(User{2, "Bob", "bob@example.com"});

    // Invoke: 每次调用执行自定义逻辑
    EXPECT_CALL(mock, FindById(_))
        .Times(3)
        .willRepeatedly(::testing::Invoke([&results](int) mutable {
            if (results.empty()) return std::optional<User>{};
            auto r = results.front();
            results.pop();
            return r;
        }));

    auto r1 = mock.FindById(1); // Alice
    auto r2 = mock.FindById(2); // nullopt
    auto r3 = mock.FindById(3); // Bob

    EXPECT_TRUE(r1.has_value());
    EXPECT_FALSE(r2.has_value());
    EXPECT_TRUE(r3.has_value());
}

```

5.7 Mock 完整速查

Mock 功能	语法
定义 Mock 方法	MOCK_METHOD(ret, name, (args), (cv))
期望调用	EXPECT_CALL(obj, method(args))
默认行为	ON_CALL(obj, method(args))
返回固定值	.willOnce(Return(val))
返回引用	.willOnce(ReturnRef(var))
多次调用返回不同值	.willOnce(...).willOnce(...)
始终返回	.willRepeatedly(Return(val))
调用次数	.Times(n) / .Times(AtLeast(n))
无调用期望	.Times(0)
任意参数	_


```
// | 自定义行为 | .willOnce(Invoke(Lambda)) |
// | 参数匹配 | Ge(), Lt(), AllOf(), AnyOf() |
// | 检查调用 | Mock::VerifyAndClearExpectations(&m) |
// |_____|
```

6. 参数化测试

6.1 基本参数化测试

```
// 被测函数
bool IsPalindrome(const std::string& str) {
    if (str.empty()) return true;
    size_t left = 0, right = str.size() - 1;
    while (left < right) {
        if (str[left] != str[right]) return false;
        ++left; --right;
    }
    return true;
}

// 参数化测试 Fixture
class PalindromeTest
: public ::testing::TestWithParam<std::pair<std::string, bool>> {
    // 自定义参数打印（可选，失败时更易读）
    static std::string ParamName(
        const ::testing::TestParamInfo<ParamType>& info) {
        return info.param.first;
    }
};

// 测试用例
TEST_P(PalindromeTest, ShouldCorrectlyIdentifyPalindrome) {
    auto [input, expected] = GetParam();
    EXPECT_EQ(IsPalindrome(input), expected)
        << "Failed for input: \"" << input << "\"";
}

// 注册测试参数
INSTANTIATE_TEST_SUITE_P(
    PalindromeTests,
    PalindromeTest,
    ::testing::Values(
        std::pair{"", true},
        std::pair{"a", true},
        std::pair{"aa", true},
        std::pair{"ab", false},
        std::pair{"aba", true},
        std::pair{"abc", false},
        std::pair{"racecar", true},
        std::pair{"hello", false},
        std::pair{"A man a plan a canal Panama", false} // 大小写+空格
```

```
)  
);
```

6.2 多参数组合测试

```
// 被测: 计算器  
class Calculator {  
public:  
    int Add(int a, int b) const { return a + b; }  
    int Subtract(int a, int b) const { return a - b; }  
    int Multiply(int a, int b) const { return a * b; }  
    double Divide(int a, int b) const {  
        if (b == 0) throw std::invalid_argument("division by zero");  
        return static_cast<double>(a) / b;  
    }  
};  
  
// 参数结构  
struct CalcTestCase {  
    int a;  
    int b;  
    int expectedSum;  
    int expectedDiff;  
    int expectedProd;  
    double expectedQuot;  
};  
  
class CalculatorParamTest  
    : public ::testing::TestWithParam<CalcTestCase> {  
protected:  
    Calculator calc_;  
};  
  
TEST_P(CalculatorParamTest, AllOperations) {  
    auto [a, b, sum, diff, prod, quot] = GetParam();  
  
    EXPECT_EQ(calc_.Add(a, b), sum) << "Add: " << a << "+" << b;  
    EXPECT_EQ(calc_.Subtract(a, b), diff) << "Sub: " << a << "-" << b;  
    EXPECT_EQ(calc_.Multiply(a, b), prod) << "Mul: " << a << "*" << b;  
    EXPECT_DOUBLE_EQ(calc_.Divide(a, b), quot)  
        << "Div: " << a << "/" << b;  
}  
  
INSTANTIATE_TEST_SUITE_P(  
    BasicOps,  
    CalculatorParamTest,  
    ::testing::Values(  
        CalcTestCase{2, 3, 5, -1, 6, 2.0/3.0},  
        CalcTestCase{0, 0, 0, 0, 0, 0}, // 0/0 会异常, 这个用例会失败  
        CalcTestCase{10, 5, 15, 5, 50, 2.0},  
        CalcTestCase{-5, 3, -2, -8, -15, -5.0/3.0}  
    )  
);
```

```
);
```

6.3 组合参数 (Combine)

```
// 使用 Combine 生成笛卡尔积
class CombineTest : public ::testing::TestWithParam<
    std::tuple<int, int, char>> {
};

TEST_P(CombineTest, FuzzTest) {
    auto [a, b, op] = GetParam();
    // 对 a 和 b 的所有组合测试所有操作
    calculator calc;
    switch (op) {
        case '+': calc.Add(a, b); break;
        case '-': calc.Subtract(a, b); break;
        case '*': calc.Multiply(a, b); break;
    }
}

INSTANTIATE_TEST_SUITE_P(
    Fuzzing,
    CombineTest,
    ::testing::Combine(
        ::testing::Values(0, 1, -1, 100, -100), // a
        ::testing::Values(0, 1, -1, 100, -100), // b
        ::testing::Values('+', '-', '*')        // op
    )
    // 生成 5*5*3 = 75 个测试用例
);
```

7. 覆盖率

7.1 什么是代码覆盖率

覆盖率衡量测试执行了多少代码。提供量化指标，但不等于质量。

指标	含义	示例
行覆盖率	执行了多少行代码	if (x>0) a(); b(); — x>0 时覆盖 100%
分支覆盖率	执行了多少分支	上面例子中 else 分支未覆盖 → 50%
函数覆盖率	调用了多少函数	10 个函数调用了 8 个 → 80%
条件覆盖率	每个 boolean 子条件的 true/false	if (a && b) — 需要 a=T/F, b=T/F
MC/DC	Modified Condition/Decision Coverage	安全关键系统要求，每个条件独立影响结果

示例：代码覆盖率和分支覆盖率的区别

	行覆盖	分支覆盖
<code>void f(int x) {</code>		
<code>if (x > 0) {</code>	✓	✓ (true 分支)
<code>doA();</code>	✓	
<code>} else {</code>	✗	✗ (false 分支)
<code>doB();</code>	✗	
<code>}</code>		
<code>}</code>		

测试：f(1) → 行覆盖 60%，分支覆盖 50%

7.2 gcov / lcov 使用

```
# 1. 编译时添加覆盖率标志
g++ -fprofile-arcs -ftest-coverage -g -O0 \
    -o my_test my_test.cpp -lgtest -lgtest_main -pthread
# 或用 CMake
# target_compile_options(my_lib PRIVATE --coverage)
# target_link_options(my_test PRIVATE --coverage)

# 2. 运行测试
./my_test

# 3. 生成覆盖率报告
gcov my_source.cpp                # 生成 my_source.cpp.gcov 文本文件

# 4. 可视化 (lcov + genhtml)
lcov --capture --directory . \
    --output-file coverage.info
lcov --remove coverage.info \
    '/usr/*' '*/tests/*' '*/gtest/*' \
    --output-file coverage_filtered.info
genhtml coverage_filtered.info \
    --output-directory coverage_report

# 5. 查看报告
# 打开 coverage_report/index.html

# 精确到分支覆盖率
lcov --rc lcov_branch_coverage=1 \
    --capture --directory . --output-file coverage.info
genhtml coverage.info --output-directory coverage_report \
    --branch-coverage
```

7.3 CMake 集成覆盖率

```
# CMakeLists.txt 中添加覆盖率选项
option(ENABLE_COVERAGE "Enable code coverage" OFF)

if(ENABLE_COVERAGE)
    if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
```

```

        add_compile_options(--coverage)
        add_link_options(--coverage)
    endif()
endif()

# 添加覆盖率目标
add_custom_target(coverage
    COMMAND ${CMAKE_CTEST_COMMAND}
    COMMAND lcov --capture --directory . --output-file coverage.info
    COMMAND lcov --remove coverage.info
        '*/tests/*' '*/third_party/*' '/usr/*'
    --output-file coverage_filtered.info
    COMMAND genhtml coverage_filtered.info
    --output-directory coverage_report
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
    COMMENT "Generating coverage report"
)

```

7.4 覆盖率目标

- **目标覆盖率**：通常 80%+ 行覆盖率是合理目标
- **不要迷信 100%**：100% 覆盖率 != 没有 Bug
- **关注未覆盖代码**：未覆盖的代码是测试盲区
- **新代码**：新功能应该接近 90%+
- **关键路径**：支付、安全相关必须 100%
- **防御性代码**：`default: assert(false)` 之类的可能很难覆盖

8. CI 集成

8.1 CTest 基础

```

# CMakeLists.txt
enable_testing()

add_test(NAME UnitTests COMMAND my_tests)
add_test(NAME IntegrationTests COMMAND integration_tests)

# 设置测试属性
set_tests_properties(UnitTests PROPERTIES
    TIMEOUT 30
    LABELS "unit"
)

set_tests_properties(IntegrationTests PROPERTIES
    TIMEOUT 120
    LABELS "integration;slow"
)

# 使用 Google Test 的自动发现
include(GoogleTest)

```

```
gtest_discover_tests(my_tests
  DISCOVERY_TIMEOUT 10
  TEST_FILTER_SUFFIX "DISABLED_"
)
```

```
# CTest 命令
ctest                # 运行所有测试
ctest -N             # 列出但不运行
ctest -R "User.*"    # 运行匹配的测试
ctest -E "Slow.*"    # 排除匹配的测试
ctest -L "unit"      # 按标签运行
ctest --output-on-failure # 失败时输出详细信息
ctest -j4            # 并行运行
ctest --repeat-until-fail 10 # 重复直到失败（排查 flaky test）
ctest --rerun-failed  # 只重跑失败的测试
```

8.2 GitHub Actions 集成

```
# .github/workflows/ci.yml
name: C++ CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-22.04

    strategy:
      matrix:
        build_type: [Debug, Release]
        compiler: [g++-12, clang++-15]

    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          submodules: recursive

      - name: Install Dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y cmake g++-12 clang-15 lcov

      - name: Configure
        run: |
          cmake -B build \
            -DCMAKE_BUILD_TYPE=${{ matrix.build_type }} \
            -DCMAKE_CXX_COMPILER=${{ matrix.compiler }} \
```

```

-DENABLE_COVERAGE=ON

- name: Build
  run: cmake --build build -j$(nproc)

- name: Test
  working-directory: build
  run: ctest --output-on-failure -j$(nproc)

- name: Coverage (Debug only)
  if: matrix.build_type == 'Debug'
  working-directory: build
  run: |
    lcov --capture --directory . --output-file coverage.info
    lcov --remove coverage.info '/usr/*' '*/tests/*' \
      --output-file coverage_filtered.info
    genhtml coverage_filtered.info --output-directory coverage_html

- name: Upload Coverage
  if: matrix.build_type == 'Debug'
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report-${{ matrix.compiler }}
    path: build/coverage_html/

```

8.3 Jenkins Pipeline (声明式)

```

// Jenkinsfile
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Configure') {
      steps {
        sh '''cmake -B build \
          -DCMAKE_BUILD_TYPE=Debug \
          -DENABLE_COVERAGE=ON
          ...
        '''
      }
    }

    stage('Build') {
      steps {
        sh 'cmake --build build -j$(nproc)'
      }
    }
  }
}

```

```

stage('Test') {
    steps {
        dir('build') {
            sh 'ctest --output-on-failure -j$(nproc)'
        }
    }
    post {
        always {
            // 收集测试报告
            junit 'build/test_results/*.xml'
        }
    }
}

stage('Coverage') {
    steps {
        dir('build') {
            sh '''
                lcov --capture --directory . \
                    --output-file coverage.info
                lcov --remove coverage.info \\\
                    '/usr/*' '*/tests/*' \\\
                    --output-file coverage_filtered.info
                genhtml coverage_filtered.info \
                    --output-directory coverage_html
            '''
        }
    }
    post {
        always {
            publishHTML(target: [
                allowMissing: true,
                reportName: 'Coverage Report',
                reportDir: 'build/coverage_html',
                reportFiles: 'index.html'
            ])
        }
    }
}
}

```

9. 测试最佳实践

9.1 AAA 模式 (Arrange-Act-Assert)

```

// Arrange-Act-Assert: 每个测试遵循三段式结构
TEST(OrderServiceTest, ShouldApplyDiscount_whenVipCustomer) {
    // Arrange (准备): 创建对象, 设置前置条件
    auto mockRepo = std::make_shared<MockOrderRepository>();
}

```



```

auto mockNotifier = std::make_shared<MockNotifier>();
OrderService service(mockRepo, mockNotifier);

Order order{1, 100.0, true}; // VIP 订单
EXPECT_CALL(*mockRepo, Save(order)).WillOnce(Return(true));

// Act (执行): 调用被测方法
bool result = service.ProcessOrder(order);

// Assert (断言): 验证结果
EXPECT_TRUE(result);
// 注: Mock 的 EXPECT_CALL 自动验证调用
}

```

9.2 测试命名

```

// 推荐格式: Should_ExpectedBehavior_when_StateUnderTest
TEST(CalculatorTest, ShouldReturnSum_whenTwoPositiveNumbers);
TEST(CalculatorTest, ShouldThrowException_whenDividingByZero);
TEST(LoggerTest, ShouldNotCrash_whenFileCannotBeOpened);
TEST(ParserTest, ShouldReturnEmpty_whenInputIsEmpty);
TEST(ValidatorTest, ShouldRejectEmail_whenMissingAtSign);

// 也可以按被测方法分类
TEST(UserServiceTest, GetDisplayName_ReturnsFormattedName_whenUserExists);
TEST(UserServiceTest, GetDisplayName_ReturnsUnknown_whenUserNotFound);

```

9.3 每次测试一个行为

```

// BAD - 一个测试验证多个行为
TEST(StackTest, VerifyEverything) {
    Stack s;
    EXPECT_TRUE(s.IsEmpty());
    EXPECT_EQ(s.Size(), 0);

    s.Push(1);
    EXPECT_FALSE(s.IsEmpty());
    EXPECT_EQ(s.Size(), 1);
    EXPECT_EQ(s.Pop(), 1);

    s.Push(2); s.Push(3);
    EXPECT_EQ(s.Top(), 3);
}

// 问题: 第一步失败后所有后续断言都不可信

// GOOD - 每个测试一个清晰的场景
TEST(StackTest, ShouldBeEmpty_whenConstructed) {
    Stack s;
    EXPECT_TRUE(s.IsEmpty());
    EXPECT_EQ(s.Size(), 0);
}

```

```

TEST(StackTest, ShouldNotBeEmpty_AfterPush) {
    Stack s;
    s.Push(1);
    EXPECT_FALSE(s.IsEmpty());
}

TEST(StackTest, ShouldReturnLastPushedValue_whenPop) {
    Stack s;
    s.Push(1);
    s.Push(2);
    EXPECT_EQ(s.Pop(), 2);
}

```

9.4 边界测试

```

// 每个方法都必须测边界条件
TEST(StringUtilTest, ShouldHandleEmptyString) {
    EXPECT_TRUE(IsPalindrome("")); // 空串
}

TEST(StringUtilTest, ShouldHandleSingleChar) {
    EXPECT_TRUE(IsPalindrome("a")); // 单字符
    EXPECT_TRUE(IsPalindrome("z"));
}

TEST(StringUtilTest, ShouldHandleMaxSize) {
    std::string huge(1000000, 'a'); // 极大输入
    EXPECT_TRUE(IsPalindrome(huge));
}

TEST(StringUtilTest, ShouldHandleNullptr) {
    EXPECT_THROW(ProcessString(nullptr), std::invalid_argument);
}

TEST(StringUtilTest, ShouldHandleBoundaryValues) {
    // 整数的边界
    EXPECT_NO_THROW(Divide(std::numeric_limits<int>::max(), 1));
    EXPECT_THROW(Divide(std::numeric_limits<int>::min(), 0),
                  std::invalid_argument);
}

```

9.5 测试行为，而非实现

```

// BAD - 测试实现细节（私有方法、内部状态）
TEST(SorterTest, ShouldCallSwapThreeTimes) {
    // 不应该测内部 swap 次数，这是实现细节
}

// GOOD - 测试可观察的行为
TEST(SorterTest, ShouldProduceSortedOutput_whenUnsortedInput) {

```

```

std::vector<int> input = {5, 2, 8, 1, 9};
std::vector<int> expected = {1, 2, 5, 8, 9};

Sort(input);

EXPECT_EQ(input, expected);
}

```

9.6 依赖注入使代码可测试

```

// BAD - 硬编码依赖, 不可测
class ReportGenerator {
public:
    void Generate() {
        MySQLConnection conn;           // 硬编码依赖
        auto data = conn.Query("...");
        // ... 很难 Mock
    }
};

// GOOD - 依赖注入, 可测
class ReportGenerator {
public:
    explicit ReportGenerator(std::unique_ptr<IDatabase> db)
        : db_(std::move(db)) {}
    void Generate() {
        auto data = db_>Query("..."); // 可注入 Mock
    }
private:
    std::unique_ptr<IDatabase> db_;
};

```

9.7 什么应该测试/什么不应该测试

应该测试	不应该测试
业务逻辑 (算法、计算)	简单的 getter/setter
边界条件	框架代码 (如 Spring Boot 启动)
错误处理路径	标准库的功能
数据转换	不会失败的代码
条件分支	配置文件
状态转换	纯数据对象 (POJO/POCO)
多线程交互 (难但重要)	日志输出格式

9.8 Flaky Test (不稳定测试) 处理

```
// Flaky test: 同样的代码，有时通过有时失败

// 常见原因和解决方案：

// 1. 时间相关
// BAD
TEST(TimerTest, TimesOut) {
    auto start = clock::now();
    service.Wait(100ms);
    auto elapsed = clock::now() - start;
    EXPECT_LT(elapsed, 150ms);    // 可能受系统负载影响
}

// GOOD - 使用 mock clock
TEST(TimerTest, TimesOut) {
    MockClock clock;
    clock.SetNow(TimePoint{0});
    // 控制时间流逝
}

// 2. 全局状态/单例
// BAD - 测试顺序影响结果
// GOOD - 每个测试后恢复初始状态
class DatabaseTest : public ::testing::Test {
    void TearDown() override {
        Database::ResetToDefault();    // 清理全局状态
    }
};

// 3. 随机数
// BAD - 使用真实随机
// GOOD - 注入可控制的随机数生成器
```

9.9 测试结构模板总结

```
// =====
// 单元测试文件模板
// =====

#include <gtest/gtest.h>
#include <gmock/gmock.h>

// 前向声明
class MyServiceTest : public ::testing::Test {
protected:
    void SetUp() override {
        // 每个测试前的初始化
    }
    void TearDown() override {
        // 每个测试后的清理
    }
};
```

```

    }
    // 测试辅助方法
};

// 命名: Should_ExpectedBehavior_when_Condition
TEST_F(MyServiceTest, ShouldReturnX_whenValidInput) {
    // Arrange
    // Act
    // Assert
}

TEST_F(MyServiceTest, ShouldThrowException_whenInvalidInput) {
    // Arrange
    // Act & Assert
}

// 参数化测试
class MyServiceParamTest : public ::testing::TestWithParam<ParamType> {};
TEST_P(MyServiceParamTest, ...) { }
INSTANTIATE_TEST_SUITE_P(/* ... */);

```

10. 面试核心

面试重点

1. **AAA 模式**: Arrange-Act-Assert, 每段只能写这一种代码
2. **FIRST 原则**: 什么是好的单元测试
3. **测试金字塔**: Unit > Integration > E2E 的比例和原因
4. **Mock vs Stub**: Mock 关心行为 (是否调用), Stub 只返回预设值
5. **依赖注入与可测试性**: 代码难测试 → 设计有问题
6. **覆盖率**: 行覆盖、分支覆盖的区别, 覆盖率不是银弹
7. **TDD 红-绿-重构**: 先写测试, 再写代码

高频考点

- **什么时候不写单元测试**: 探索性代码、生命周期短的脚本、原型
- **好的测试命名**: `Should_xxx_when_xxx` 格式
- **EXPECT vs ASSERT**: 失败后继续 vs 立即停止
- **SetUp vs SetUpTestCase**: 每个测试 vs 整个 suite
- **代码覆盖率 100% 有意义吗**: 没有! 可以通过没有断言的测试达到 100%
- **私有方法要不要测**: 通过公共接口间接测试, 不应该直接测私有方法
- **如何测试多线程代码**: 困难但可以通过模拟时间、注入调度器等方法测试
- **GMock 的 EXPECT_CALL vs ON_CALL**: 期望 vs 默认行为

常见陷阱

1. **不写测试** — "代码太简单不需要测试" → 简单代码最容易被无意识地破坏
2. **测试实现而非行为** — 重构内部实现时测试全红
3. **测试间有依赖** — 全局状态、单例导致测试顺序敏感
4. **慢测试** — 单元测试应该毫秒级，集成测试才配秒级
5. **不覆盖边界/异常** — 只测 Happy Path
6. **Mock 太多** — Mock 太多说明类职责过多 (SRP 问题)
7. **断言不足** — 有 Mock 但没有 ASSERT，测试永远绿
8. **测试代码质量差** — 测试也是产品代码，也需要可读性和 DRY
9. **忽略 Flaky Test** — "偶尔失败没事" → 习以为常后真失败看不出来
10. **覆盖率为指标考核** — 逆激励：开发者为了覆盖率写无用测试

建议背诵内容

1. **FIRST 原则**: "Fast, Independent, Repeatable, Self-validating, Timely"
2. **AAA 三段式**: "Arrange (准备) → Act (执行) → Assert (验证)"
3. **测试金字塔**: "单元测试多而快，集成测试少而适中，E2E 更少而慢"
4. **TDD 三步**: "红 — 写失败测试，绿 — 最小实现，重构 — 改善结构"
5. **Mock 核心三行**:

```
class MockX : public IX {  
    MOCK_METHOD(RetType, Method, (Args), (override));  
};  
EXPECT_CALL(mock, Method(_)).Times(1).WillOnce(Return(value));
```

6. **覆盖率的真相**: "高覆盖率不等于高质量，低覆盖率一定意味着测试不足"
7. **测试命名黄金法则**: "Should_ExpectedBehavior_When_Condition"
8. **依赖注入金句**: "如果你的类很难测试，说明它的设计有问题。测试困难是坏设计的检测器。"

面试建议: 单元测试面试题往往是"你平时写单元测试吗？怎么写？"这种开放性问题。用具体的代码示例 (AAA、Mock、Fixture) 和实际的覆盖率数字来回答，远比背诵概念有说服力。准备一个真实的测试案例，从被测代码到测试代码完整讲述。